// GUIDA GRATIS

UN LOOP VALE QUANTO LA SUA *verifica.*

Un agente AI si ferma quando il lavoro sembra finito. Questa guida ti insegna a dargli un controllo che può eseguire davvero, su misura per la tua app, con un esempio eseguito ora e il codice per rifarlo.

1 · Mappa	le funzioni e cosa vuol dire "fatto"	passo 01
2 · Flussi	il percorso di un utente vero	passo 02
3 · Test	un controllo che guida l'app	passo 03
4 · Prova	screenshot e registrazioni	passo 04
5 · Blocco	il loop non chiude senza prova	passo 05
6 · Aggiorna	la mappa cresce col codice	passo 06

// COSA TI SERVE

Un'app web (anche piccola) e un assistente AI per il codice (nella guida usiamo Claude Code). Per il passo 03 usiamo Playwright, gratuito e open source: playwright.dev

// COME USARLA

Ogni passo ha: **cosa fai**, **il prompt da incollare**, un **esempio vero** eseguito per questa guida (su un mini negozio inventato) e **un consiglio**.

// PASSO 01 · LA MAPPA

01 MAPPA OGNI *funzione.*

Se l'agente non sa cosa significa "fatto", decide da solo. Una scheda per funzione dice stati, punti di contatto e criteri di superamento.

01 Un file per funzione
Esempio: checkout.md. Una funzione = un file, scritto dal punto di vista dell'utente, non del codice.

02 Gli stati
Vuoto, compilato, in pagamento, pagato, fallito. Ogni stato in cui l'utente può trovarsi.

03 I punti di contatto
Pulsante carrello, form carta, campo promo: tutto ciò che l'utente tocca.

04 Passa quando
3-5 frasi verificabili: "l'ordine è salvato", "parte l'email di ricevuta", "il carrello si svuota".

// PROMPT DA INCOLLARE

«Leggi il codice della funzione [nome] e scrivi **skill/features/[nome].md** con: stati, punti di contatto, e "passa quando" in frasi che un utente capirebbe. Non descrivere il codice.»

// RISPOSTA (estratto, mini negozio di esempio)

```
STATI: vuoto · compilato · in pagamento · pagato · fallito
PUNTI DI CONTATTO: pulsante carrello · campo promo · form carta
PASSA QUANDO: ordine salvato · carrello vuoto dopo · messaggio "Ordine pagato"
```

> **Consiglio —** rileggi i "passa quando" come cliente: se uno non lo potresti controllare a occhio, riscrivilo.

// PASSO 02 · I FLUSSI

02 TRACCIA IL PERCORSO *dell'utente.*

La mappa dice cosa esiste. Il diagramma di flusso dice come ci arrivano gli utenti: i bug veri stanno tra una funzione e l'altra.

01 Scegli un percorso
Registrazione → onboarding → primo documento → condivisione. Un percorso per file.

02 Un passo, una prova
Dopo ogni passo scrivi cosa deve risultare vero: "account esiste", "workspace pronto".

03 Le deviazioni
Cosa succede se salta un passo ("oppure salta")? Anche i percorsi alternativi vanno scritti.

04 Salvalo come riferimento
Mettilo accanto alla mappa nella skill di verifica: l'agente lo rilegge a ogni giro.

// PROMPT DA INCOLLARE

«Scrivi **skill/flows/[nome].md**: i passi del percorso [descrivilo], per ogni passo cosa deve risultare vero, e le deviazioni possibili. Chiedimi cosa manca.»

// RISPOSTA (estratto, esempio)

```
1. Aggiungi la tazza al carrello → vero: carrello visibile
2. Applica SAVE10 → vero: totale 18,00 €
3. Paga → vero: messaggio "Ordine pagato"
4. Fine → vero: ordine salvato
DEVIAZIONE: carta non valida → resta nel carrello
```

Consiglio — i passi 3 e 4 spesso sembrano riusciti a schermo ma non lo sono: scrivi sempre la prova sotto la schermata.

// PASSO 03 · IL TEST SU MISURA

03 FAI USARE L'APP *come un utente.*

I test verdi dicono che il codice gira, non che il checkout funziona. Un controllo costruito per la tua app apre il browser e fa le stesse azioni di una persona. Strumento: Playwright (github.com/microsoft/playwright), gratuito.

01 Installa
pip install playwright, poi playwright install chromium. Due comandi, una volta.

02 Scrivi il percorso
Una riga per azione (clicca, scrivi, paga) e una per ogni "passa quando" della mappa.

03 Stampa PASS o FAIL
Una riga per criterio, così l'agente legge un esito netto e non deve interpretare.

04 Lancialo ora
python verifica.py: guarda cosa esce prima di fidarti.

// ESEMPIO ESEGUITO ORA — verifica.py sul mini negozio (dati inventati)

```
$ python verifica.py
PASS carrello aperto
PASS promo applicato (18,00 €)
PASS pagamento riuscito
FAIL ordine salvato
```

// COSA HA TROVATO

Il pagamento sembrava riuscito (messaggio "Ordine pagato"), ma l'ordine **non veniva salvato**. Un test sul solo codice non l'avrebbe visto: il controllo a misura sì.

> **Consiglio —** non copiare il mio test: chiedi all'assistente di scriverne uno per il TUO percorso, partendo dal file della mappa.

// PASSO 04 · LA PROVA

04 PRETENDI LA *prova.*

"Fatto" senza prova è solo una frase. Ogni giro del loop deve lasciare qualcosa che puoi guardare: uno screenshot, una registrazione, l'output del test.

01 Screenshot a ogni passo
Nel test aggiungi pg.screenshot(path="...") dopo le azioni chiave.

02 Una cartella per giro
prove/giro-01/, prove/giro-02/: così confronti prima e dopo.

03 Il rapporto in 5 righe
Cosa è stato fatto, cosa è passato, cosa no, dove sono le prove.

04 Opzionale: la registrazione
Playwright può registrare un video della sessione (record_video_dir) quando serve.

// PROMPT DA INCOLLARE

«Dopo ogni modifica lancia verifica.py, salva gli screenshot in prove/giro-NN/ e scrivi un rapporto di 5 righe: **fatto, passato, fallito, dove sono le prove**. Non dire "fatto" se un criterio è FAIL.»

// ESEMPIO (rapporto, dal test eseguito ora)

Fatto: aggiunto codice promo. **Passato:** carrello, promo, pagamento. **Fallito:** ordine salvato. **Prove:** prove/giro-01/prova-checkout.png

> **Consiglio —** guarda tu lo screenshot almeno una volta: è il modo più veloce per accorgerti che il test controlla la cosa giusta.

// PASSO 05-06 · BLOCCO E AGGIORNAMENTO

05 IL LOOP FINISCE QUANDO *la prova passa.*

L'agente non deve decidere da solo di aver finito: un blocco lo ferma finché tutti i criteri sono PASS. E la mappa va aggiornata a ogni funzione nuova.

01 Il blocco
Regola nelle istruzioni: "non chiudere il lavoro finché verifica.py non stampa solo PASS".

02 Se un criterio fallisce
L'agente corregge e rilancia. Massimo 3 giri, poi si ferma e ti avvisa.

03 Aggiorna la mappa
Ogni funzione nuova o modificata: prima il file in skill/features/, poi il codice.

04 Controlla a fine settimana
Una funzione senza scheda è una funzione che nessuno sta verificando.

Playwright (strumento)	github.com/microsoft/playwright	passo 03
Playwright MCP (collega il browser all'assistente)	github.com/microsoft/playwright-mcp	opzionale
Documentazione	playwright.dev/python/docs/intro	passi 03-04

// RICORDA

Link controllati il 28/10/2026. Il controllo copre solo ciò che hai scritto nella mappa: non sostituisce una revisione tua sulle parti delicate (pagamenti, dati dei clienti). Il test d'esempio è su un negozio inventato.